Mastering Microservices Architecture -- Design, Patterns, and Enterprise Implementation Guide

---

Author: Senior Software Architect **Target Audience:** Java developers with 3+ years experience aiming for architect-level microservices expertise **Prerequisites:** Core Java, Spring Boot, SQL, REST APIs, basic Docker knowledge

---

Table of Contents

1. Microservices Architecture Fundamentals
2. Complete Microservices System Architecture
3. Microservices Core Components
4. Service Discovery Deep Dive
5. API Gateway Architecture
6. Microservices Security Architecture
7. Inter-Service Communication
8. Event-Driven Architecture
9. Distributed Data Management
10. Caching Architecture
11. Microservices Design Patterns
12. Resilience and Fault Tolerance
13. Microservices Deployment Architecture
14. Microservices Observability
15. Production Challenges in Microservices
16. Complete Sample Microservices Project
17. Full Project Structures
18. Microservices Best Practices
19. Interview Questions (100+)
20. Hands-on Exercises

Part 1: Microservices Architecture Fundamentals

1.1 What is Microservices Architecture?

Microservices architecture is an approach where an application is built as a collection of **small, independent, loosely coupled services**, each responsible for a specific business capability, running in its own process, and communicating over lightweight protocols (typically HTTP/REST or messaging).

```
Monolith:
+-----------------------------------------------------+
|                Single Application                    |
|  +----------+ +----------+ +----------+            |
|  | User     | | Order    | | Payment  |            |
|  | Module   | | Module   | | Module   |            |
|  +----------+ +----------+ +----------+            |
|  +----------+ +----------+ +----------+            |
|  | Inventory| | Notif.   | | Reporting|            |
|  | Module   | | Module   | | Module   |            |
|  +----------+ +----------+ +----------+            |
|           Single Database                            |
+-----------------------------------------------------+

Microservices:
+--------+  +--------+  +---------+  +-----------+
| User   |  | Order  |  | Payment |  | Inventory |
| Service|  | Service|  | Service |  | Service   |
+--------+  +--------+  +---------+  +-----------+
| UserDB |  | OrderDB|  |PaymentDB|  |InvDB      |
+--------+  +--------+  +---------+  +-----------+
     |           |            |            |
     +-----------+------------+------------+
                Message Broker (Kafka)
```

1.2 Monolith vs Microservices

Aspect	Monolith	Microservices
Deployment	Deploy entire application	Deploy individual services
Scaling	Scale entire application	Scale specific services

Technology	Single tech stack	Polyglot (different languages per service)
Database	Single shared database	Database per service
Team structure	Large team on one codebase	Small teams per service
Failure impact	One bug can crash everything	Failure isolated to one service
Development speed	Slow (large codebase)	Fast (small, focused codebases)
Testing	End-to-end testing complex	Unit/integration testing per service
Communication	In-process method calls	Network calls (HTTP, messaging)
Complexity	Simpler architecture	Complex distributed system

1.3 Advantages of Microservices

1. **Independent deployment** -- Update payment service without touching user service
2. **Scalability** -- Scale only the service that needs it (e.g., scale Order Service during sale events)
3. **Technology diversity** -- Java for business logic, Python for ML, Node.js for real-time
4. **Fault isolation** -- If notification service crashes, ordering still works
5. **Team autonomy** -- Each team owns their service end-to-end
6. **Faster development** -- Small codebases are easier to understand and modify

1.4 Disadvantages of Microservices

1. **Distributed system complexity** -- Network failures, latency, data consistency
2. **Operational overhead** -- Many services to deploy, monitor, and manage
3. **Data management** -- No simple JOINs across service databases
4. **Testing complexity** -- Integration testing across services is harder
5. **Debugging difficulty** -- Tracing requests across multiple services
6. **Increased infrastructure cost** -- More servers, containers, load balancers

1.5 When NOT to Use Microservices

Scenario	Why Monolith is Better
Small team (< 5 developers)	Microservices overhead exceeds benefits
Simple domain	No need for independent scaling
Startup / MVP	Speed to market matters more

Unclear domain boundaries	Bad service boundaries cause constant refactoring
No DevOps maturity	Need CI/CD, monitoring, container orchestration

1.6 Real-World Examples

System	Why Microservices
E-commerce	Product catalog, orders, payments, inventory all scale independently
Banking	Account service, transaction service, fraud detection need isolation
Netflix	1000+ services; each team owns their domain
Uber	Ride matching, pricing, payments, notifications are separate services
Government systems	Eligibility, enrollment, notices are separate bounded contexts

---

Part 2: Complete Microservices System Architecture

2.1 Enterprise Architecture Diagram

```
+-------------------------------------------------------------------+
|                          CLIENT APPLICATIONS                      |
|  (Web App, Mobile App, Third-party Integrations)                  |
+-------------------------------------------------------------------+
                              |
                              v
+-------------------------------------------------------------------+
|                           LOAD BALANCER                           |
|                        (Nginx / AWS ALB)                          |
+-------------------------------------------------------------------+
                              |
                              v
+-------------------------------------------------------------------+
|                           API GATEWAY                             |
|                     (Spring Cloud Gateway)                        |
|                                                                   |
|  Responsibilities:                                                |
|  - Routing          - Authentication     - Rate limiting          |
|  - Load balancing   - Request logging    - Response caching       |
+-------------------------------------------------------------------+
         |              |              |              |
         v              v              v              v
+----------+    +----------+    +----------+    +----------+
| Auth     |    | User     |    | Order    |    | Product  |
| Service  |    | Service  |    | Service  |    | Service  |
| (JWT)    |    |          |    |          |    |          |
+----------+    +----------+    +----------+    +----------+
| AuthDB   |    | UserDB   |    | OrderDB  |    |ProductDB |
+----------+    +----------+    +----------+    +----------+
                                     |
         +---------------------------+---------------------------+
         |                           |                           |
         v                           v                           v
+----------+               +----------+               +----------+
| Payment  |               | Inventory|               | Notif.   |
| Service  |               | Service  |               | Service  |
+----------+               +----------+               +----------+
```

```
|PaymentDB |                |  InvDB   |                |  NoSQL   |
+----------+                +----------+                +----------+

Cross-Cutting Concerns:
+------------------+  +------------------+  +------------------+
| Service Discovery|  | Config Server    |  | Message Broker   |
| (Eureka)         |  | (Spring Cloud    |  | (Kafka)          |
|                  |  |  Config)         |  |                  |
+------------------+  +------------------+  +------------------+
+------------------+  +------------------+  +------------------+
| Cache Layer      |  | Distributed      |  | Monitoring       |
| (Redis)          |  | Tracing (Zipkin) |  | (Prometheus +    |
|                  |  |                  |  |  Grafana)        |
+------------------+  +------------------+  +------------------+
```

2.2 How Components Interact

Flow	Description
Client -> API Gateway	All requests enter through a single entry point
API Gateway -> Auth Service	Validates JWT token for every request
API Gateway -> Business Services	Routes requests to appropriate service
Service -> Service Discovery	Each service registers itself and discovers others
Service -> Config Server	Fetches externalized configuration
Service -> Message Broker	Publishes/consumes events asynchronously
Service -> Cache	Reads/writes frequently accessed data
Service -> Database	Each service has its own dedicated database
Monitoring tools	Collect logs, metrics, and traces from all services

---

Part 3: Microservices Core Components

3.1 API Gateway

Aspect	Details
What	Single entry point for all client requests
Why	Centralizes cross-cutting concerns (auth, routing, rate limiting)
When	Always, in any microservices architecture
How	Spring Cloud Gateway, Kong, AWS API Gateway
Where	Between clients and internal services

3.2 Service Discovery

Aspect	Details
What	Registry where services register themselves and find other services
Why	Services have dynamic IPs in cloud/container environments
When	Always, unless using Kubernetes native service discovery
How	Netflix Eureka, HashiCorp Consul, Kubernetes Services
Where	Central registry accessible by all services

3.3 Config Server

Aspect	Details
What	Centralized configuration management for all services
Why	Avoid hardcoding config; change config without redeployment
When	Always, especially in multi-environment setups (dev/staging/prod)
How	Spring Cloud Config backed by Git repository
Where	Central server that all services connect to on startup

3.4 Authentication Service

Aspect	Details
What	Handles user authentication and JWT token generation
Why	Centralized identity management, single sign-on
When	Always, in any secured system
How	Spring Security + JWT + OAuth2
Where	Dedicated service called by API Gateway for token validation

3.5 Message Broker

Aspect	Details
What	Middleware for asynchronous communication between services
Why	Decouples services, enables event-driven architecture, handles spikes
When	For async operations (notifications, analytics, eventual consistency)
How	Apache Kafka, RabbitMQ, AWS SQS
Where	Central cluster accessible by all producing/consuming services

3.6 Cache Layer

Aspect	Details
What	In-memory data store for frequently accessed data
Why	Reduces database load, improves response time (sub-millisecond)
When	For read-heavy data: product catalogs, user sessions, config data
How	Redis, Memcached
Where	Between application layer and database

3.7 Monitoring System

Aspect	Details
What	Collects logs, metrics, and traces from all services
Why	Visibility into distributed system health and performance
When	Always, non-negotiable in production
How	Prometheus + Grafana (metrics), ELK (logs), Zipkin/Jaeger (traces)
Where	Centralized monitoring infrastructure

3.8 Database Per Service

Aspect	Details
What	Each microservice owns its own database (separate schema or instance)
Why	Data independence, loose coupling, independent scaling
When	Always in true microservices architecture
How	Each service configures its own DataSource
Where	Dedicated databases (User DB, Order DB, Payment DB, etc.)

Part 4: Service Discovery Deep Dive

4.1 Why Service Discovery?

In cloud/container environments, service instances have **dynamic IP addresses**. They scale up/down, restart, and move between hosts. Service discovery solves: "How does Order Service find Payment Service?"

```
WITHOUT Service Discovery:
  Order Service --> http://192.168.1.45:8082/api/payments  (Hardcoded! Breaks on
restart)

WITH Service Discovery:
  Order Service --> "payment-service"  (Logical name resolved dynamically)
```

4.2 Netflix Eureka

```
+-------------------+
|   Eureka Server   |  (Service Registry)
|  (port 8761)      |
+-------------------+
   ^     ^     ^
   |     |     |
   | Register  |
   |     |     |
+------+ +------+ +------+
| User | | Order| | Pay  |
| Svc  | | Svc  | | Svc  |
+------+ +------+ +------+

Flow:
1. Each service REGISTERS with Eureka on startup
2. Each service sends HEARTBEAT every 30s
3. When Order Service needs Payment Service:
   - Asks Eureka for "payment-service" instances
   - Gets list: [192.168.1.45:8082, 192.168.1.46:8082]
   - Client-side load balancer picks one
```

Eureka Server Setup

```java
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

```yaml
# Eureka Server application.yml
server:
  port: 8761
eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
```

Service Registration

```yaml
# Any microservice application.yml
spring:
  application:
    name: order-service
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true
```

4.3 Kubernetes Service Discovery

In Kubernetes, service discovery is built-in:

```yaml
# Kubernetes Service
apiVersion: v1
```

```yaml
kind: Service
metadata:
  name: payment-service
spec:
  selector:
    app: payment-service
  ports:
    - port: 8080
      targetPort: 8080
# Other services call: http://payment-service:8080/api/payments
# Kubernetes DNS resolves the service name automatically
```

Part 5: API Gateway Architecture

5.1 Responsibilities

```
Client Request
     |
     v
+--------------------------------------------------+
|                   API GATEWAY                    |
|                                                   |
|  1. ROUTING                                      |
|     /api/users/**    --> user-service             |
|     /api/orders/**   --> order-service            |
|     /api/products/** --> product-service          |
|                                                   |
|  2. AUTHENTICATION                               |
|     Validate JWT token on every request           |
|                                                   |
|  3. RATE LIMITING                                |
|     Max 100 requests/minute per client            |
|                                                   |
|  4. LOAD BALANCING                               |
|     Distribute requests across service instances  |
|                                                   |
|  5. RESPONSE CACHING                             |
|     Cache GET responses for static data           |
|                                                   |
|  6. REQUEST/RESPONSE LOGGING                     |
|     Log all incoming requests for audit           |
+--------------------------------------------------+
```

5.2 Spring Cloud Gateway Implementation

```java
@SpringBootApplication
public class ApiGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApiGatewayApplication.class, args);
```

```
    }
}
```

```yaml
# application.yml
server:
  port: 8080

spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://user-service
          predicates:
            - Path=/api/users/**
          filters:
            - StripPrefix=0

        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=0

        - id: product-service
          uri: lb://product-service
          predicates:
            - Path=/api/products/**
          filters:
            - StripPrefix=0

        - id: payment-service
          uri: lb://payment-service
          predicates:
            - Path=/api/payments/**
          filters:
            - StripPrefix=0
```

5.3 Custom Authentication Filter

```java
@Component
public class JwtAuthenticationFilter implements
GatewayFilterFactory<JwtAuthenticationFilter.Config> {

    @Autowired
    private JwtUtil jwtUtil;

    @Override
    public GatewayFilter apply(Config config) {
        return (exchange, chain) -> {
            String path = exchange.getRequest().getURI().getPath();

            // Skip auth for public endpoints
            if (isPublicEndpoint(path)) {
                return chain.filter(exchange);
            }

            String authHeader = exchange.getRequest().getHeaders()
                .getFirst(HttpHeaders.AUTHORIZATION);

            if (authHeader == null || !authHeader.startsWith("Bearer ")) {
                exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
                return exchange.getResponse().setComplete();
            }

            String token = authHeader.substring(7);
            try {
                Claims claims = jwtUtil.validateToken(token);
                // Add user info to headers for downstream services
                ServerHttpRequest modifiedRequest = exchange.getRequest()
                    .mutate()
                    .header("X-User-Id", claims.getSubject())
                    .header("X-User-Role", claims.get("role", String.class))
                    .build();
                return
chain.filter(exchange.mutate().request(modifiedRequest).build());
            } catch (JwtException e) {
                exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
                return exchange.getResponse().setComplete();
            }
        };
    }

    private boolean isPublicEndpoint(String path) {
        return path.startsWith("/api/auth/login") ||
               path.startsWith("/api/auth/register") ||
```

```
                path.startsWith("/actuator");
    }

    public static class Config { }
}
```

5.4 Rate Limiting

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://user-service
          predicates:
            - Path=/api/users/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10
                redis-rate-limiter.burstCapacity: 20
                key-resolver: "#{@userKeyResolver}"
```

```java
@Bean
public KeyResolver userKeyResolver() {
    return exchange -> Mono.just(
        exchange.getRequest().getRemoteAddress().getAddress().getHostAddress()
    );
}
```

Part 6: Microservices Security Architecture

6.1 JWT-Based Authentication Flow

```
1. Client sends credentials:
   POST /api/auth/login
   { "email": "user@mail.com", "password": "secret" }

2. Auth Service validates credentials:
   - Check user in database
   - Verify password hash
   - Generate JWT token with claims (userId, role, expiry)

3. Auth Service returns JWT:
   { "token": "eyJhbGciOiJIUzI1NiIs..." }

4. Client sends JWT with every request:
   GET /api/orders
   Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

5. API Gateway validates JWT:
   - Verify signature
   - Check expiration
   - Extract user info
   - Forward to downstream service with user headers

6. Downstream service trusts the headers:
   X-User-Id: 42
   X-User-Role: ADMIN
```

6.2 JWT Token Structure

```
Header:    { "alg": "HS256", "typ": "JWT" }
Payload:   { "sub": "42", "email": "user@mail.com",
             "role": "ADMIN", "iat": 1709812345, "exp": 1709898745 }
Signature: HMACSHA256(base64(header) + "." + base64(payload), secret)
```

6.3 Auth Service Implementation

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired private AuthService authService;

    @PostMapping("/register")
    public ResponseEntity<UserDTO> register(@Valid @RequestBody RegisterRequest
req) {
        return ResponseEntity.status(201).body(authService.register(req));
    }

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest
req) {
        return ResponseEntity.ok(authService.login(req));
    }

    @PostMapping("/refresh")
    public ResponseEntity<AuthResponse> refresh(@RequestBody RefreshRequest
req) {
        return
ResponseEntity.ok(authService.refreshToken(req.getRefreshToken()));
    }
}

@Service
public class AuthService {

    @Autowired private UserRepository userRepository;
    @Autowired private PasswordEncoder passwordEncoder;
    @Autowired private JwtUtil jwtUtil;

    public AuthResponse login(LoginRequest request) {
        User user = userRepository.findByEmail(request.getEmail())
            .orElseThrow(() -> new AuthException("Invalid credentials"));

        if (!passwordEncoder.matches(request.getPassword(),
user.getPasswordHash())) {
            throw new AuthException("Invalid credentials");
        }

        String accessToken = jwtUtil.generateToken(user.getId(),
```

```
            user.getEmail(), user.getRole().name());
        String refreshToken = jwtUtil.generateRefreshToken(user.getId());

        return new AuthResponse(accessToken, refreshToken, 86400);
    }
}
```

6.4 OAuth2 Integration

```
OAuth2 Flow (Third-party login):

1. Client -> Auth Service: "Login with Google"
2. Auth Service -> Google: Redirect user to Google login
3. User logs in on Google
4. Google -> Auth Service: Authorization code
5. Auth Service -> Google: Exchange code for user info
6. Auth Service creates/updates local user
7. Auth Service -> Client: JWT token

This enables:
- "Login with Google/GitHub/Microsoft"
- Single Sign-On (SSO)
- Delegated authentication
```

6.5 Role-Based Access Control (RBAC)

```
// Custom annotation for role checking
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface RequiresRole {
    String[] value();
}

// Interceptor in each microservice
@Component
public class RoleAuthorizationInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request,
                             HttpServletResponse response,
                             Object handler) {
```

```
        if (handler instanceof HandlerMethod method) {
            RequiresRole annotation =
method.getMethodAnnotation(RequiresRole.class);
            if (annotation != null) {
                String userRole = request.getHeader("X-User-Role");
                if (!Arrays.asList(annotation.value()).contains(userRole)) {
                    response.setStatus(HttpStatus.FORBIDDEN.value());
                    return false;
                }
            }
        }
        return true;
    }
}

// Usage in controller
@GetMapping("/admin/reports")
@RequiresRole({"ADMIN", "MANAGER"})
public List<Report> getReports() { ... }
```

Part 7: Inter-Service Communication

7.1 Synchronous Communication (REST)

```
Order Service  ----HTTP GET---->  Payment Service
               <---Response----
               (Blocking, waits for response)
```

Using OpenFeign (Declarative REST Client)

```java
// Payment Service Feign Client (defined in Order Service)
@FeignClient(name = "payment-service",
             fallback = PaymentServiceFallback.class)
public interface PaymentServiceClient {

    @PostMapping("/api/payments")
    PaymentResponse processPayment(@RequestBody PaymentRequest request);

    @GetMapping("/api/payments/{orderId}")
    PaymentResponse getPaymentByOrderId(@PathVariable Long orderId);
}

// Usage in Order Service
@Service
public class OrderService {

    @Autowired
    private PaymentServiceClient paymentClient;

    @Transactional
    public Order placeOrder(OrderRequest request) {
        Order order = orderRepository.save(new Order(request));

        // Synchronous call to Payment Service
        PaymentResponse payment = paymentClient.processPayment(
            new PaymentRequest(order.getId(), order.getTotalAmount()));

        if ("SUCCESS".equals(payment.getStatus())) {
            order.setStatus(OrderStatus.CONFIRMED);
        } else {
```

```
            order.setStatus(OrderStatus.PAYMENT_FAILED);
        }
        return orderRepository.save(order);
    }
}

// Fallback for circuit breaker
@Component
public class PaymentServiceFallback implements PaymentServiceClient {
    @Override
    public PaymentResponse processPayment(PaymentRequest request) {
        return new PaymentResponse("PENDING", "Payment service unavailable");
    }

    @Override
    public PaymentResponse getPaymentByOrderId(Long orderId) {
        return new PaymentResponse("UNKNOWN", "Payment service unavailable");
    }
}
```

7.2 Asynchronous Communication (Messaging)

```
Order Service  --publish event-->  [Kafka Topic]  --consume-->  Notification
Service
               (Non-blocking, fire and forget)                  Inventory
Service
                                                                Analytics
Service
```

Kafka Producer (Order Service)

```
@Service
public class OrderEventPublisher {

    @Autowired
    private KafkaTemplate<String, OrderEvent> kafkaTemplate;

    public void publishOrderCreated(Order order) {
        OrderEvent event = new OrderEvent(
            order.getId(), "ORDER_CREATED",
            order.getCustomerId(), order.getTotalAmount(),
            Instant.now());
```

```
        kafkaTemplate.send("order-events", order.getId().toString(), event);
    }
}
```

Kafka Consumer (Notification Service)

```
@Service
public class OrderEventConsumer {

    @Autowired
    private NotificationService notificationService;

    @KafkaListener(topics = "order-events", groupId = "notification-group")
    public void handleOrderEvent(OrderEvent event) {
        switch (event.getType()) {
            case "ORDER_CREATED" ->

notificationService.sendOrderConfirmation(event.getCustomerId());
            case "ORDER_SHIPPED" ->

notificationService.sendShippingNotification(event.getCustomerId());
            case "ORDER_DELIVERED" ->

notificationService.sendDeliveryConfirmation(event.getCustomerId());
        }
    }
}
```

7.3 When to Use Synchronous vs Asynchronous

Criteria	Synchronous (REST/Feign)	Asynchronous (Kafka/RabbitMQ)
Response needed	Immediately	Not immediately
Coupling	Tight (caller waits)	Loose (fire and forget)
Failure handling	Caller must handle	Retry from queue
Use case	Get payment status	Send notification
Consistency	Strong	Eventual

Performance	Blocked	Non-blocking
Example	Check inventory before order	Publish order event for analytics

---

Part 8: Event-Driven Architecture

8.1 What is Event-Driven Architecture?

A pattern where services communicate by producing and consuming **events** rather than making direct calls. An event represents something that happened: "Order was created", "Payment was processed", "Inventory was updated".

Why Event-Driven Over REST Calls

REST (Synchronous)	Event-Driven (Asynchronous)
Order Service CALLS Inventory directly	Order Service PUBLISHES event, doesn't know consumers
If Inventory is down, Order fails	Order succeeds immediately, Inventory processes later
Adding a new consumer requires changing Order Service	New consumers subscribe without any changes
Tight coupling between services	Loose coupling -- services don't know about each other

When to Use Event-Driven Architecture

- **Multiple services** need to react to the same business event
- Services can tolerate **eventual consistency** (not instant)
- You need **audit trail** of all events (event log)
- System needs to handle **traffic spikes** (events buffered in queue)

When NOT to Use

- Simple CRUD applications with 2-3 services
- When you need **immediate consistency** across services
- When the added complexity of Kafka/RabbitMQ is not justified

```
+----------+                            +----------+
| Order    |  --OrderCreatedEvent-->    | Inventory|
| Service  |                            | Service  |
| (Producer|  --OrderCreatedEvent-->    +----------+
|          |                            +----------+
```

```
+----------+  --OrderCreatedEvent-->    | Notif.   |
                                        | Service  |
              [Kafka Topic:             +----------+
               order-events]            +----------+
                                        | Analytics|
                                        | Service  |
                                        +----------+

Key difference from REST:
- REST: Order Service CALLS Inventory Service directly (coupled)
- Events: Order Service PUBLISHES event, doesn't know who consumes it
(decoupled)
```

8.2 Kafka Architecture

```
+---------+     +---------+     +---------+
|Producer1|     |Producer2|     |Producer3|
|(Order)  |     |(Payment)|     |(User)   |
+---------+     +---------+     +---------+
     |               |               |
     v               v               v
+-----------------------------------------------+
|                  KAFKA CLUSTER                |
|                                                |
|  Topic: order-events                          |
|  +----------+ +----------+ +----------+       |
|  |Partition0| |Partition1| |Partition2|       |
|  +----------+ +----------+ +----------+       |
|                                                |
|  Topic: payment-events                        |
|  +----------+ +----------+                    |
|  |Partition0| |Partition1|                    |
|  +----------+ +----------+                    |
+-----------------------------------------------+
     |               |               |
     v               v               v
+---------+     +---------+     +---------+
|Consumer1|     |Consumer2|     |Consumer3|
|(Notif.) |     |(Invent.)|     |(Analyt.)|
+---------+     +---------+     +---------+
```

8.3 Event Classes

```java
// Base event
public abstract class BaseEvent {
    private String eventId;
    private String eventType;
    private Instant timestamp;
    private String source;
}

// Order events
public class OrderCreatedEvent extends BaseEvent {
    private Long orderId;
    private Long customerId;
    private List<OrderItemDTO> items;
    private BigDecimal totalAmount;
    private String shippingAddress;
}

public class OrderCancelledEvent extends BaseEvent {
    private Long orderId;
    private String reason;
}

// Payment events
public class PaymentProcessedEvent extends BaseEvent {
    private Long paymentId;
    private Long orderId;
    private BigDecimal amount;
    private String status; // SUCCESS, FAILED
    private String transactionId;
}

// Inventory events
public class InventoryUpdatedEvent extends BaseEvent {
    private Long productId;
    private int previousQuantity;
    private int newQuantity;
    private String updateReason; // ORDER, RESTOCK, ADJUSTMENT
}
```

8.4 Kafka Configuration

```yaml
# application.yml
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer:
org.springframework.kafka.support.serializer.JsonSerializer
      acks: all
      retries: 3
    consumer:
      group-id: ${spring.application.name}
      auto-offset-reset: earliest
      key-deserializer:
org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer:
org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: "com.app.event"
```

8.5 Real-World Event Flow: Order Placement

```
Step 1: Client places order
        POST /api/orders { items, address, paymentMethod }

Step 2: Order Service creates order (status: PENDING)
        Publishes: OrderCreatedEvent to "order-events"

Step 3: Inventory Service consumes OrderCreatedEvent
        Reserves stock for each item
        Publishes: InventoryReservedEvent to "inventory-events"

Step 4: Payment Service consumes OrderCreatedEvent
        Charges customer's payment method
        Publishes: PaymentProcessedEvent to "payment-events"

Step 5: Order Service consumes PaymentProcessedEvent
        If SUCCESS: updates order status to CONFIRMED
        If FAILED: updates order status to PAYMENT_FAILED
                   Publishes: OrderCancelledEvent
```

```
Step 6: Notification Service consumes all events
        Sends email/SMS for each status change
```

Part 9: Distributed Data Management

Why Distributed Data is Hard

In a monolith, all data lives in one database -- you can use JOINs, foreign keys, and multi-table transactions. In microservices, each service has its own database. This creates three fundamental challenges:

Challenge	Monolith Solution	Microservices Solution
Cross-service queries	SQL JOIN	API calls + data denormalization
Multi-service transactions	Single DB transaction	Saga pattern
Data consistency	ACID guarantees	Eventual consistency + event sourcing

> ***CAP Theorem Reminder:*** *In a distributed system, you can only guarantee 2 of 3: Consistency, Availability, Partition Tolerance. Most microservices choose AP (Available + Partition Tolerant) and accept eventual consistency.*

9.1 Database Per Service Pattern

Each microservice owns its data and exposes it only through its API. No direct cross-service database access.

```
+---------+     +---------+     +---------+
| User    |     | Order   |     | Product |
| Service |     | Service |     | Service |
+---------+     +---------+     +---------+
     |               |               |
+---------+     +---------+     +---------+
| UserDB  |     | OrderDB |     |ProductDB|
|PostgreSQL|    | MySQL   |     |MongoDB  |
+---------+     +---------+     +---------+

Rules:
1. Only User Service accesses UserDB
2. Order Service CANNOT query UserDB directly
3. If Order Service needs user info, it calls User Service API
4. Each service can use different database technology
```

9.2 Challenges

Distributed Transactions

In monolith: single database transaction across all tables. In microservices: order creation + payment + inventory involve 3 different databases.

Data Consistency

Without shared DB, you can't use JOINs or foreign keys across services. Data may be temporarily inconsistent (eventual consistency).

9.3 Saga Pattern

What is the Saga Pattern?

A sequence of local transactions where each service publishes events to trigger the next step. If one step fails, compensating transactions undo previous steps.

Why Sagas Over Distributed Transactions (2PC)

Two-Phase Commit (2PC)	Saga Pattern
Locks resources across services	No distributed locks
Single coordinator failure = system freeze	Each service acts independently
Not scalable	Scales with number of services
Tight coupling	Loose coupling

Two Approaches

Choreography	Orchestration
Services react to events independently	Central coordinator directs each step
No single point of failure	Easier to understand and debug
Harder to track overall progress	Coordinator can track saga state
Best for simple sagas (3-4 steps)	Best for complex sagas (5+ steps)

Choreography-Based Saga

```
Order Service         Inventory Service      Payment Service
     |                       |                      |
     | OrderCreated          |                      |
     +----------------------->|                      |
     |                       | StockReserved        |
     |                       +---------------------->|
     |                       |                      | PaymentProcessed
     |<----------------------+----------------------+
     | Update to CONFIRMED   |                      |
     |                       |                      |

FAILURE (Payment fails):
     |                       |                      |
     |                       |        PaymentFailed |
     |                       |<---------------------+
     |                       | Release stock        |
     | OrderCancelled        | (Compensating txn)   |
     |<----------------------+                      |
```

Orchestration-Based Saga

```java
@Service
public class OrderSagaOrchestrator {

    @Autowired private OrderRepository orderRepository;
    @Autowired private InventoryServiceClient inventoryClient;
    @Autowired private PaymentServiceClient paymentClient;
    @Autowired private NotificationServiceClient notificationClient;

    @Transactional
    public OrderResult executeOrderSaga(OrderRequest request) {
        // Step 1: Create order
        Order order = orderRepository.save(new Order(request,
OrderStatus.PENDING));

        try {
            // Step 2: Reserve inventory
            InventoryResponse inv = inventoryClient.reserve(order.getItems());
            if (!inv.isSuccess()) {
                order.setStatus(OrderStatus.FAILED);
                order.setFailureReason("Insufficient stock");
                return OrderResult.failed(order);
            }
```

```
            // Step 3: Process payment
            PaymentResponse pay = paymentClient.process(
                new PaymentRequest(order.getId(), order.getTotalAmount()));
            if (!pay.isSuccess()) {
                // Compensate: release inventory
                inventoryClient.release(order.getItems());
                order.setStatus(OrderStatus.PAYMENT_FAILED);
                return OrderResult.failed(order);
            }

            // Step 4: Confirm order
            order.setStatus(OrderStatus.CONFIRMED);
            order.setPaymentId(pay.getTransactionId());
            orderRepository.save(order);

            // Step 5: Notify customer
            notificationClient.sendConfirmation(order.getCustomerId(),
order.getId());

            return OrderResult.success(order);

        } catch (Exception e) {
            // Compensate all previous steps
            compensate(order);
            throw new OrderProcessingException("Saga failed", e);
        }
    }

    private void compensate(Order order) {
        try { inventoryClient.release(order.getItems()); } catch (Exception
ignored) {}
        try { paymentClient.refund(order.getId()); } catch (Exception ignored)
{}
        order.setStatus(OrderStatus.FAILED);
        orderRepository.save(order);
    }
}
```

9.4 CQRS (Command Query Responsibility Segregation)

What is CQRS?

Separate the write model (commands) from the read model (queries). Use different data stores, schemas, and even programming models for reads and writes.

Why CQRS?

- **Reads and writes have different performance requirements** -- reads need fast denormalized data, writes need normalized integrity
- **Read-heavy systems** (100 reads per write) benefit from optimized read stores
- **Complex queries** can use search indexes (Elasticsearch) while writes use relational DB

When NOT to Use CQRS

- Simple CRUD applications -- the complexity is not justified
- When strong consistency is required (CQRS introduces eventual consistency)
- Small datasets where a single database handles both read/write efficiently

```
Commands (Write):                    Queries (Read):
POST /api/orders                     GET /api/orders?status=PENDING
PUT /api/orders/{id}/cancel          GET /api/orders/dashboard

+----------+     Publish      +-----------+
| Command  | --- Event -----> | Query     |
| Service  |                  | Service   |
| (Write)  |                  | (Read)    |
+----------+                  +-----------+
| Normalized|                 |Denormalized|
| PostgreSQL|                 |MongoDB /   |
|           |                 |Elasticsearch|
+-----------+                 +-----------+
```

9.5 Event Sourcing

Instead of storing current state, store all events that led to the current state.

```
Traditional: Account table row -> { id: 1, balance: 500 }

Event Sourcing: Event log
  Event 1: AccountCreated  { amount: 0 }
  Event 2: MoneyDeposited  { amount: 1000 }
  Event 3: MoneyWithdrawn  { amount: 300 }
  Event 4: MoneyWithdrawn  { amount: 200 }
  Current state: 0 + 1000 - 300 - 200 = 500

Benefits:
- Complete audit trail
```

```
- Can rebuild state at any point in time
- Can replay events to debug issues
```

Part 10: Caching Architecture

What is Caching in Microservices?

Caching stores frequently accessed data in fast storage (Redis, in-memory) to reduce database load and improve response times.

Why Caching is Critical

In a microservices architecture, a single user request may trigger 3-5 inter-service calls. Without caching, each call hits its database, creating a cascade of queries. Caching can reduce response time from **200ms to 2ms**.

Cache Strategy Decision Guide

Strategy	How It Works	Use When
Cache-Aside	App checks cache, on miss loads from DB and writes to cache	Read-heavy, tolerates stale data
Write-Through	App writes to cache AND DB simultaneously	Need immediate cache consistency
Write-Behind	App writes to cache, async writes to DB	Write-heavy, tolerates data loss risk
Read-Through	Cache itself loads data from DB on miss	Simplified app logic

When NOT to Use Caching

- **Frequently changing data** -- cache becomes stale immediately
- **Unique queries** -- every query is different, cache hit rate is near 0%
- When **data consistency** is critical (financial balances)

10.1 Why Caching in Microservices?

```
WITHOUT Cache:
  Client -> API -> Service -> Database (every time)
  Response time: 50-200ms

WITH Cache:
```

```
Client -> API -> Service -> Redis (cache hit)
Response time: 1-5ms

Client -> API -> Service -> Redis (cache miss) -> Database
Response time: 50-200ms (first time only)
```

10.2 Cache Strategies

Cache-Aside (Lazy Loading)

```java
@Service
public class ProductService {

    @Autowired private ProductRepository productRepository;
    @Autowired private RedisTemplate<String, Product> redisTemplate;

    public Product getProduct(Long id) {
        String key = "product:" + id;

        // 1. Check cache first
        Product cached = redisTemplate.opsForValue().get(key);
        if (cached != null) {
            return cached; // Cache hit
        }

        // 2. Cache miss -- load from database
        Product product = productRepository.findById(id)
            .orElseThrow(() -> new NotFoundException("Product not found"));

        // 3. Store in cache for future requests
        redisTemplate.opsForValue().set(key, product, Duration.ofMinutes(30));

        return product;
    }

    // Invalidate cache on update
    public Product updateProduct(Long id, ProductUpdateRequest request) {
        Product product = productRepository.findById(id).orElseThrow();
        product.setName(request.getName());
        product.setPrice(request.getPrice());
        Product saved = productRepository.save(product);

        // Invalidate cache
        redisTemplate.delete("product:" + id);
```

```
        return saved;
    }
}
```

Write-Through Cache

```
// Data is written to cache AND database simultaneously
public Product saveProduct(Product product) {
    Product saved = productRepository.save(product);    // Write to DB
    redisTemplate.opsForValue().set(                     // Write to cache
        "product:" + saved.getId(), saved, Duration.ofHours(1));
    return saved;
}
```

Write-Behind Cache (Write-Back)

```
Data written to cache first, then asynchronously persisted to database.
- Faster writes (cache is in-memory)
- Risk of data loss if cache crashes before DB write
- Used for high-throughput write scenarios
```

10.3 Spring Boot + Redis Configuration

```yaml
spring:
  data:
    redis:
      host: localhost
      port: 6379
      timeout: 2000ms
  cache:
    type: redis
    redis:
      time-to-live: 3600000  # 1 hour in ms
```

```java
@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory factory) {
        RedisCacheConfiguration config =
RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(30))
            .disableCachingNullValues()
            .serializeValuesWith(RedisSerializationContext.SerializationPair
                .fromSerializer(new GenericJackson2JsonRedisSerializer()));

        return RedisCacheManager.builder(factory)
            .cacheDefaults(config)
            .withCacheConfiguration("products",
                RedisCacheConfiguration.defaultCacheConfig()
                    .entryTtl(Duration.ofHours(1)))
            .withCacheConfiguration("users",
                RedisCacheConfiguration.defaultCacheConfig()
                    .entryTtl(Duration.ofMinutes(15)))
            .build();
    }
}

// Usage with @Cacheable
@Service
public class ProductService {

    @Cacheable(value = "products", key = "#id")
    public Product getProduct(Long id) {
        return productRepository.findById(id).orElseThrow();
    }

    @CacheEvict(value = "products", key = "#id")
    public void updateProduct(Long id, ProductUpdateRequest request) {
        Product product = productRepository.findById(id).orElseThrow();
        product.setName(request.getName());
        productRepository.save(product);
    }

    @CacheEvict(value = "products", allEntries = true)
    public void clearProductCache() { }
}
```

Part 11: Microservices Design Patterns

Overview of Key Patterns

Microservices patterns solve recurring problems in distributed systems. Understanding WHEN to use each pattern is as important as understanding HOW it works.

Pattern	Problem It Solves
API Gateway	Single entry point for all clients
Circuit Breaker	Prevent cascade failures when a service is down
Bulkhead	Isolate failures so one slow service doesn't kill all
Saga	Distributed transactions across services
Strangler Fig	Gradual migration from monolith to microservices
Sidecar	Cross-cutting concerns (logging, mTLS) without code changes

11.1 API Gateway Pattern

Already covered in Part 5. Single entry point for all clients, handles routing, auth, rate limiting.

11.2 Circuit Breaker Pattern

Problem: If Payment Service is down, Order Service keeps sending requests, wasting resources and degrading user experience.

Solution: Circuit breaker detects failures and stops calling the failing service temporarily.

```
States:
  CLOSED  --> Requests flow normally
              If failure threshold exceeded --> OPEN

  OPEN    --> All requests fail immediately (fast fail)
              After wait duration --> HALF_OPEN

  HALF_OPEN -> Allow limited requests to test recovery
               If success --> CLOSED
               If failure --> OPEN

Timeline:
```

```
  |---CLOSED---|---OPEN---|---HALF_OPEN---|---CLOSED---|
  normal calls   fast fail   test calls     back to normal
```

11.3 Bulkhead Pattern

Problem: One slow service consumes all threads, blocking other services.

Solution: Isolate thread pools per service call.

```
WITHOUT Bulkhead:
  [Thread Pool: 200 threads]
  All service calls share same pool
  If Payment Service is slow -> all 200 threads blocked -> entire app hangs

WITH Bulkhead:
  [Pool A: 50 threads] -> Payment Service calls
  [Pool B: 50 threads] -> Inventory Service calls
  [Pool C: 100 threads] -> Other calls
  If Payment Service is slow -> only 50 threads blocked -> rest works fine
```

11.4 Saga Pattern

Already covered in Part 9. Manages distributed transactions across services.

11.5 Strangler Pattern

Problem: How to migrate from monolith to microservices without a big-bang rewrite.

Solution: Gradually replace monolith functionality with microservices.

```
Phase 1: All traffic goes to monolith
  Client -> [Monolith with all features]

Phase 2: New features built as microservices, old ones still in monolith
  Client -> [API Gateway]
                |
         +------+------+
         |             |
    [New user     [Monolith
     service]      (orders, payments, etc.)]

Phase 3: Gradually move features out
```

```
  Client -> [API Gateway]
         |         |         |
    [User Svc] [Order Svc] [Monolith (remaining)]

Phase 4: Monolith fully replaced
  Client -> [API Gateway]
         |         |         |
    [User Svc] [Order Svc] [Payment Svc]
```

11.6 Sidecar Pattern

Deploy helper functionality alongside the main service in the same container/pod.

```
+-----------------------------------+
| Pod / Host                        |
|  +-------------+  +------------+  |
|  | Main        |  | Sidecar    |  |
|  | Application |  | (Envoy     |  |
|  | (Java)      |  |  proxy)    |  |
|  +-------------+  +------------+  |
|        |                |         |
|        +---local net----+         |
+-----------------------------------+
Sidecar handles:
  - mTLS between services
  - Logging agent
  - Health checking
  - Traffic management
```

Part 12: Resilience and Fault Tolerance

Why Resilience Matters in Microservices

In a monolith, failures are typically all-or-nothing. In microservices, **partial failures** are common -- one service may be down while others work fine. Resilience patterns ensure the entire system doesn't collapse when one service fails.

The Four Pillars of Resilience

Pattern	What It Does	Real-World Analogy
Circuit Breaker	Stops calling a failing service	Electrical circuit breaker
Retry	Automatically retries failed calls	Redial a phone call
Rate Limiter	Limits how many calls per second	Traffic speed limit
Timeout	Sets maximum wait time for a response	Restaurant wait time limit

Resilience4j -- The Standard Library

Resilience4j is the recommended library for resilience in Spring Boot 3+. It replaces Netflix Hystrix (which is deprecated).

12.1 Resilience4j Circuit Breaker

```
<dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
```

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        registerHealthIndicator: true
        slidingWindowSize: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 10s
```

```
        permittedNumberOfCallsInHalfOpenState: 3
        slidingWindowType: COUNT_BASED
```

```
@Service
public class OrderService {

    @CircuitBreaker(name = "paymentService", fallbackMethod =
"paymentFallback")
    public PaymentResponse processPayment(PaymentRequest request) {
        return paymentServiceClient.process(request);
    }

    // Fallback when circuit is open
    private PaymentResponse paymentFallback(PaymentRequest request, Throwable
t) {
        log.warn("Payment service unavailable, queuing for later: {}",
t.getMessage());
        paymentQueue.enqueue(request); // Queue for retry later
        return new PaymentResponse("PENDING", "Payment queued for processing");
    }
}
```

12.2 Retry

```
resilience4j:
  retry:
    instances:
      inventoryService:
        maxAttempts: 3
        waitDuration: 500ms
        retryExceptions:
          - java.io.IOException
          - java.net.SocketTimeoutException
        ignoreExceptions:
          - com.app.exception.BusinessException
```

```
@Retry(name = "inventoryService", fallbackMethod = "inventoryFallback")
public InventoryResponse checkStock(Long productId) {
```

```
    return inventoryClient.getStock(productId);
}
```

12.3 Rate Limiter

```yaml
resilience4j:
  ratelimiter:
    instances:
      externalApi:
        limitForPeriod: 50
        limitRefreshPeriod: 1s
        timeoutDuration: 500ms
```

```
@RateLimiter(name = "externalApi")
public ExternalData callExternalApi(String param) {
    return externalApiClient.getData(param);
}
```

12.4 Timeout

```
@TimeLimiter(name = "paymentService", fallbackMethod = "timeoutFallback")
public CompletableFuture<PaymentResponse> processPaymentAsync(PaymentRequest
request) {
    return CompletableFuture.supplyAsync(() -> paymentClient.process(request));
}
```

```yaml
resilience4j:
  timelimiter:
    instances:
      paymentService:
        timeoutDuration: 3s
        cancelRunningFuture: true
```

12.5 Combining Patterns

```
// Order of execution: Retry -> CircuitBreaker -> RateLimiter -> TimeLimiter
@CircuitBreaker(name = "paymentService", fallbackMethod = "fallback")
@Retry(name = "paymentService")
@RateLimiter(name = "paymentService")
public PaymentResponse processPayment(PaymentRequest request) {
    return paymentServiceClient.process(request);
}
```

Part 13: Microservices Deployment Architecture

13.1 Docker

Dockerfile for Spring Boot Service

```
# Multi-stage build
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN ./mvnw package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar

# Non-root user for security
RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring

EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget -qO- http://localhost:8080/actuator/health || exit 1

ENTRYPOINT ["java", "-jar", "app.jar"]
```

13.2 Docker Compose (Development)

```
version: '3.8'
services:
  # Infrastructure
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
    ports: ["5432:5432"]
    volumes: ["postgres_data:/var/lib/postgresql/data"]
```

```yaml
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    ports: ["9092:9092"]
    environment:
      KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:29092,EXTERNAL://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  # Services
  eureka-server:
    build: ./eureka-server
    ports: ["8761:8761"]

  config-server:
    build: ./config-server
    ports: ["8888:8888"]
    depends_on: [eureka-server]

  api-gateway:
    build: ./api-gateway
    ports: ["8080:8080"]
    depends_on: [eureka-server, config-server]

  user-service:
    build: ./user-service
    depends_on: [postgres, eureka-server, config-server]
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/userdb

  order-service:
    build: ./order-service
    depends_on: [postgres, kafka, eureka-server, config-server]
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/orderdb

  payment-service:
    build: ./payment-service
    depends_on: [postgres, kafka, eureka-server, config-server]

  notification-service:
```

```yaml
    build: ./notification-service
    depends_on: [kafka, eureka-server]

volumes:
  postgres_data:
```

13.3 Kubernetes Deployment

```yaml
# Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
    spec:
      containers:
        - name: order-service
          image: myregistry/order-service:1.2.0
          ports:
            - containerPort: 8080
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
```

```yaml
              port: 8080
            initialDelaySeconds: 15
            periodSeconds: 5
          env:
            - name: SPRING_DATASOURCE_URL
              valueFrom:
                secretKeyRef:
                  name: order-db-secret
                  key: url
---
# Service
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector:
    app: order-service
  ports:
    - port: 8080
      targetPort: 8080
  type: ClusterIP
---
# HorizontalPodAutoscaler
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: order-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-service
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

Part 14: Microservices Observability

14.1 Three Pillars of Observability

```
+-------------------------------------------------------------------+
|                     OBSERVABILITY                                 |
|                                                                    |
|  +------------------+  +-----------------+  +-----------------+   |
|  |     LOGGING      |  |    METRICS      |  |   TRACING       |   |
|  |                  |  |                 |  |                 |   |
|  | What happened?   |  | How is the      |  | Where did the   |   |
|  | Error details    |  | system doing?   |  | request go?     |   |
|  |                  |  |                 |  |                 |   |
|  | Tools:           |  | Tools:          |  | Tools:          |   |
|  | ELK Stack        |  | Prometheus      |  | Zipkin          |   |
|  | (Elasticsearch,  |  | Grafana         |  | Jaeger          |   |
|  |  Logstash,       |  | Micrometer      |  | OpenTelemetry   |   |
|  |  Kibana)         |  |                 |  |                 |   |
|  +------------------+  +-----------------+  +-----------------+   |
+-------------------------------------------------------------------+
```

14.2 Centralized Logging (ELK Stack)

```
+---------+     +---------+     +---------+
| User    |     | Order   |     | Payment |
| Service |     | Service |     | Service |
| (logs)  |     | (logs)  |     | (logs)  |
+---------+     +---------+     +---------+
     |               |               |
     v               v               v
+---------------------------------------------------+
|               Logstash / Fluentd                  |
|         (Collects and transforms logs)            |
+---------------------------------------------------+
                      |
                      v
+---------------------------------------------------+
|              Elasticsearch                        |
|          (Indexes and stores logs)                |
+---------------------------------------------------+
```

```
                      |
                      v
+-----------------------------------------------+
|                 Kibana                        |
|        (Search, visualize, dashboard)         |
+-----------------------------------------------+
```

Structured Logging Configuration

```
# logback-spring.xml
<configuration>
  <appender name="JSON"
class="net.logstash.logback.appender.LogstashTcpSocketAppender">
    <destination>logstash:5000</destination>
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
      <customFields>{"service":"${spring.application.name}"}</customFields>
    </encoder>
  </appender>

  <root level="INFO">
    <appender-ref ref="JSON" />
  </root>
</configuration>
```

Correlation ID for Request Tracing

```
@Component
public class CorrelationIdFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
                         FilterChain chain) throws IOException,
ServletException {
        HttpServletRequest httpRequest = (HttpServletRequest) request;
        String correlationId = httpRequest.getHeader("X-Correlation-Id");
        if (correlationId == null) {
            correlationId = UUID.randomUUID().toString();
        }
        MDC.put("correlationId", correlationId);
        try {
            chain.doFilter(request, response);
        } finally {
```

```
            MDC.remove("correlationId");
        }
    }
}
```

14.3 Metrics (Prometheus + Grafana)

```yaml
# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health, metrics, prometheus
  metrics:
    export:
      prometheus:
        enabled: true
    tags:
      application: ${spring.application.name}
```

```java
// Custom business metrics
@Service
public class OrderService {

    private final Counter orderCounter;
    private final Timer orderProcessingTimer;

    public OrderService(MeterRegistry registry) {
        this.orderCounter = Counter.builder("orders.created")
            .tag("service", "order-service")
            .description("Total orders created")
            .register(registry);

        this.orderProcessingTimer = Timer.builder("orders.processing.time")
            .tag("service", "order-service")
            .description("Order processing duration")
            .register(registry);
    }

    public Order createOrder(OrderRequest request) {
        return orderProcessingTimer.record(() -> {
```

```
            Order order = processOrder(request);
            orderCounter.increment();
            return order;
        });
    }
}
```

Key Metrics to Monitor

Metric	Description	Alert Threshold
`http_server_requests_seconds`	Request latency	p99 > 2s
`jvm_memory_used_bytes`	JVM heap usage	> 80% of max
`hikaricp_connections_active`	Active DB connections	> 80% of pool
`system_cpu_usage`	CPU utilization	> 70% sustained
`orders.created`	Business metric: orders	Sudden drop
`http_server_requests_seconds_count`	Request rate	Sudden spike or drop

14.4 Distributed Tracing (Zipkin)

```yaml
# application.yml (Spring Boot 3+)
management:
  tracing:
    sampling:
      probability: 1.0  # Sample 100% of traces (0.1 = 10% in production)
  zipkin:
    tracing:
      endpoint: http://zipkin:9411/api/v2/spans
```

```
Trace Example: Place Order
Trace ID: abc123

  [API Gateway]        12ms
    [Order Service]      85ms
      [Inventory Svc]      15ms  (Feign call)
      [Payment Svc]        45ms  (Feign call)
      [Kafka Publish]       3ms
```

```
    [Notification Svc]     8ms   (Kafka consumer)

Zipkin UI shows this waterfall view, making it easy to identify:
- Which service is slow
- Where errors occur
- Total end-to-end latency
```

Part 15: Production Challenges in Microservices

15.1 Network Latency

Problem: Every service call adds network overhead (1-10ms per hop). A request touching 5 services accumulates 5-50ms of pure network time.

Solutions:

- Minimize synchronous call chains (prefer async events)
- Use gRPC instead of REST for internal calls (binary protocol, faster)
- Co-locate frequently communicating services
- Cache data locally to avoid repeated calls

15.2 Service Failures

Problem: In a system with 20 services, the probability of at least one being down is high.

Solutions:

Strategy	How
Circuit Breaker	Stop calling failing services, use fallback
Retry with backoff	Retry transient failures with exponential delay
Bulkhead	Isolate resources per dependency
Graceful degradation	Serve partial data if a service is down
Health checks	Kubernetes readiness/liveness probes

15.3 Distributed Debugging

Problem: A user complaint like "my order failed" could involve 7 services. Finding the root cause requires tracing across all of them.

Solutions:

- **Correlation ID:** Single request ID passed through all services
- **Distributed tracing:** Zipkin/Jaeger shows the full call chain
- **Centralized logging:** All logs in ELK, searchable by correlation ID

- **Structured logs:** JSON format with service name, trace ID, span ID

15.4 Data Consistency

Problem: Order is created but payment fails. Without shared transactions, data is inconsistent across services.

Solutions:

Pattern	Description
Saga	Compensating transactions undo failed steps
Outbox pattern	Write event to DB then publish (at-least-once delivery)
Idempotent consumers	Handle duplicate events gracefully
Eventual consistency	Accept temporary inconsistency, reconcile later

15.5 Scaling Challenges

Problem: Different services have different scaling needs. Order Service needs 10 instances during a sale; User Service needs only 2.

Solutions:

- Kubernetes HPA (Horizontal Pod Autoscaler) per service
- Scale based on custom metrics (queue depth, request rate)
- Database connection pool limits per service instance
- Stateless services for easy horizontal scaling

Part 16: Complete Sample Microservices Project

16.1 Order Processing System

Service Responsibilities

Service	Responsibility	Database	Communication
Auth Service	User registration, login, JWT generation	PostgreSQL	REST
User Service	User profiles, addresses, preferences	PostgreSQL	REST + Kafka
Product Service	Product catalog, pricing, search	PostgreSQL + Redis	REST
Order Service	Order creation, tracking, history	PostgreSQL	REST + Kafka
Inventory Service	Stock management, reservation	PostgreSQL	REST + Kafka
Payment Service	Payment processing, refunds	PostgreSQL	REST + Kafka
Notification Service	Email, SMS, push notifications	MongoDB	Kafka consumer

System Flow: Place an Order

```
1. Client: POST /api/orders { items, address, paymentMethod }
            |
2. API Gateway: Validate JWT, route to Order Service
            |
3. Order Service:
   a. Create order (PENDING) in OrderDB
   b. Call Product Service --> get item prices
   c. Call Inventory Service --> check/reserve stock
   d. Call Payment Service --> process payment
   e. If all succeed: status = CONFIRMED
   f. Publish OrderConfirmedEvent to Kafka
            |
4. Kafka Consumers:
   - Notification Service: send confirmation email
   - Analytics Service: update dashboards
   - Inventory Service: confirm stock reservation
```

API Endpoints

```
Auth Service (port 8081):
  POST  /api/auth/register       Register new user
  POST  /api/auth/login          Login, get JWT
  POST  /api/auth/refresh        Refresh JWT token

User Service (port 8082):
  GET   /api/users/{id}          Get user profile
  PUT   /api/users/{id}          Update user profile
  GET   /api/users/{id}/addresses Get user addresses

Product Service (port 8083):
  GET   /api/products            List products (paginated)
  GET   /api/products/{id}       Get product details
  GET   /api/products/search     Search products
  POST  /api/products            Create product (ADMIN)

Order Service (port 8084):
  POST  /api/orders              Place new order
  GET   /api/orders/{id}         Get order details
  GET   /api/orders/my-orders    Get current user's orders
  PUT   /api/orders/{id}/cancel  Cancel order

Inventory Service (port 8085):
  GET   /api/inventory/{productId}     Check stock
  POST  /api/inventory/reserve         Reserve stock
  POST  /api/inventory/release         Release reserved stock

Payment Service (port 8086):
  POST  /api/payments            Process payment
  GET   /api/payments/{orderId}  Get payment status
  POST  /api/payments/{id}/refund Refund payment

Notification Service (port 8087):
  (No REST APIs -- purely Kafka consumer)
```

Part 17: Full Project Structures

17.1 Standard Microservice Project Structure

```
order-service/
|
+-- src/main/java/com/app/orderservice/
|   |
|   +-- OrderServiceApplication.java
|   |
|   +-- controller/
|   |   +-- OrderController.java         // REST endpoints
|   |
|   +-- service/
|   |   +-- OrderService.java            // Business logic
|   |   +-- OrderSagaOrchestrator.java   // Saga coordination
|   |
|   +-- repository/
|   |   +-- OrderRepository.java         // JPA repository
|   |
|   +-- entity/
|   |   +-- Order.java                   // JPA entity
|   |   +-- OrderItem.java
|   |   +-- OrderStatus.java             // Enum
|   |
|   +-- dto/
|   |   +-- OrderRequest.java            // Request DTO
|   |   +-- OrderResponse.java           // Response DTO
|   |   +-- OrderItemDTO.java
|   |
|   +-- client/
|   |   +-- PaymentServiceClient.java    // Feign client
|   |   +-- InventoryServiceClient.java  // Feign client
|   |   +-- ProductServiceClient.java    // Feign client
|   |
|   +-- event/
|   |   +-- OrderCreatedEvent.java       // Kafka event
|   |   +-- OrderEventPublisher.java     // Kafka producer
|   |   +-- PaymentEventConsumer.java    // Kafka consumer
|   |
|   +-- config/
|   |   +-- KafkaConfig.java
|   |   +-- FeignConfig.java
```

```
|   |   +-- SecurityConfig.java
|   |
|   +-- exception/
|   |   +-- OrderNotFoundException.java
|   |   +-- InsufficientStockException.java
|   |   +-- GlobalExceptionHandler.java  // @ControllerAdvice
|   |
|   +-- mapper/
|       +-- OrderMapper.java             // Entity <-> DTO mapping
|
+-- src/main/resources/
|   +-- application.yml
|   +-- application-dev.yml
|   +-- application-prod.yml
|
+-- src/test/java/com/app/orderservice/
|   +-- controller/
|   |   +-- OrderControllerTest.java
|   +-- service/
|   |   +-- OrderServiceTest.java
|   +-- repository/
|       +-- OrderRepositoryTest.java
|
+-- Dockerfile
+-- pom.xml
```

17.2 Why Each Layer Exists

Layer	Purpose	Example
Controller	Handle HTTP requests, validate input, return responses	`@RestController`, `@PostMapping`
Service	Business logic, orchestration, transaction management	`@Service`, `@Transactional`
Repository	Data access abstraction	`JpaRepository<Order, Long>`
Entity	Database table mapping	`@Entity`, `@Table`
DTO	Data transfer between layers; decouple API from entities	Request/Response classes
Client	Communication with other microservices	`@FeignClient` interfaces
Event	Asynchronous event publishing and consumption	Kafka producer/consumer
Config	Spring configuration beans	`@Configuration` classes
Exception	Custom exceptions and global error handling	`@ControllerAdvice`

Mapper	Convert between entities and DTOs	Manual mapping or MapStruct

17.3 Infrastructure Services Structure

```
infrastructure/
|
+-- eureka-server/
|   +-- src/main/java/.../EurekaServerApplication.java
|   +-- src/main/resources/application.yml
|   +-- Dockerfile
|
+-- config-server/
|   +-- src/main/java/.../ConfigServerApplication.java
|   +-- src/main/resources/application.yml
|   +-- Dockerfile
|
+-- api-gateway/
|   +-- src/main/java/.../
|   |   +-- ApiGatewayApplication.java
|   |   +-- filter/JwtAuthenticationFilter.java
|   |   +-- filter/RateLimitFilter.java
|   |   +-- config/GatewayConfig.java
|   +-- src/main/resources/application.yml
|   +-- Dockerfile
|
+-- docker-compose.yml
+-- docker-compose-infra.yml  (just infra: DB, Kafka, Redis)
```

Part 18: Build It Yourself -- Order Processing Microservices

> ***Goal:*** *Build a minimal microservices system with 3 services (Order, Payment, Notification) + API Gateway + Service Discovery -- step by step.*

Concept Overview

```
Client -> API Gateway -> Order Service -> Payment Service (REST)
                                       -> Kafka -> Notification Service (consumer)
                    ^
              Eureka (Service Discovery)
```

Service	Role	Port
Eureka Server	Service registry	8761
API Gateway	Routes, JWT validation	8080
Order Service	Order CRUD, orchestrates payment	8081
Payment Service	Payment processing	8082
Notification Service	Sends emails (Kafka consumer)	8083

Step 1: Create Eureka Server (Service Discovery)

Concept: Eureka Server is a registry where services register themselves. Other services can discover and call them by name instead of hardcoded URLs.

```java
@SpringBootApplication
@EnableEurekaServer                          // Activates Eureka Server
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
```

```
    }
}
```

```yaml
# application.yml
server:
  port: 8761
eureka:
  client:
    register-with-eureka: false             # Don't register itself
    fetch-registry: false
```

Step 2: Create API Gateway

Concept: API Gateway is the single entry point. It routes requests to services by name (resolved via Eureka).

```yaml
server:
  port: 8080
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://ORDER-SERVICE             # lb:// = load-balanced via
Eureka
          predicates:
            - Path=/api/orders/**
        - id: payment-service
          uri: lb://PAYMENT-SERVICE
          predicates:
            - Path=/api/payments/**
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
```

Step 3: Create Order Service

Concept: Order Service handles order CRUD and calls Payment Service via Feign Client (service-to-service REST call).

```
// Feign Client -- calls Payment Service by name (resolved via Eureka)
@FeignClient(name = "PAYMENT-SERVICE")
public interface PaymentClient {
    @PostMapping("/api/payments")
    PaymentResponse processPayment(@RequestBody PaymentRequest request);
}

// Service
@Service
@RequiredArgsConstructor
public class OrderService {
    private final OrderRepository orderRepository;
    private final PaymentClient paymentClient;       // Feign auto-proxy
    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;

    @Transactional
    public OrderDTO createOrder(OrderRequest request) {
        Order order = orderRepository.save(new Order(request,
OrderStatus.PENDING));

        // Call Payment Service via Feign
        PaymentResponse payment = paymentClient.processPayment(
            new PaymentRequest(order.getId(), order.getTotalAmount()));

        order.setStatus(payment.isSuccess() ? OrderStatus.CONFIRMED :
OrderStatus.FAILED);

        // Publish event to Kafka for Notification Service
        kafkaTemplate.send("order-events",
            new OrderEvent(order.getId(), order.getStatus(),
request.getEmail()));

        return OrderDTO.from(order);
    }
}
```

Step 4: Create Notification Service (Kafka Consumer)

Concept: This service has NO REST APIs. It only listens to Kafka events and sends notifications.

```java
@Service
public class NotificationConsumer {

    @KafkaListener(topics = "order-events", groupId = "notification-group")
    public void handleOrderEvent(OrderEvent event) {
        if (event.getStatus() == OrderStatus.CONFIRMED) {
            emailService.sendOrderConfirmation(event.getEmail(),
event.getOrderId());
        }
        log.info("Notification sent for order: {}", event.getOrderId());
    }
}
```

Step 5: Run Everything

```
# Start in order:
1. Eureka Server    -> mvn spring-boot:run (port 8761)
2. Kafka            -> docker-compose up kafka zookeeper
3. Order Service    -> mvn spring-boot:run (port 8081)
4. Payment Service  -> mvn spring-boot:run (port 8082)
5. Notification Svc -> mvn spring-boot:run (port 8083)
6. API Gateway      -> mvn spring-boot:run (port 8080)

# Test via Gateway:
curl -X POST http://localhost:8080/api/orders \
  -H "Content-Type: application/json" \
  -d '{"items": [{"productId": 1, "qty": 2}], "email": "user@test.com"}'
```

Key Takeaways

Pattern	What It Solves
Eureka	Services discover each other by name, no hardcoded URLs
API Gateway	Single entry point, routing, cross-cutting concerns
Feign Client	Type-safe REST client for service-to-service calls
Kafka	Async event-driven communication (decoupled services)

Each service has its own DB	Data isolation, independent deployment

---

Part 18: Microservices Best Practices

18.1 API Design Best Practices

RESTful Naming Conventions

```
GOOD:
  GET    /api/orders              List orders
  GET    /api/orders/123          Get order 123
  POST   /api/orders              Create order
  PUT    /api/orders/123          Update order 123
  DELETE /api/orders/123          Delete order 123
  GET    /api/orders/123/items    Get items of order 123

BAD:
  GET  /api/getOrders
  POST /api/createOrder
  GET  /api/order/getById?id=123
```

Standard Response Format

```
public class ApiResponse<T> {
    private boolean success;
    private T data;
    private String message;
    private String errorCode;
    private Instant timestamp;
}

// Success: { "success": true, "data": { ... }, "timestamp": "..." }
// Error:   { "success": false, "message": "Order not found",
//            "errorCode": "ORDER_404", "timestamp": "..." }
```

HTTP Status Codes

Code	Meaning	Use
200	OK	Successful GET, PUT

201	Created	Successful POST
204	No Content	Successful DELETE
400	Bad Request	Invalid input
401	Unauthorized	Missing/invalid token
403	Forbidden	Insufficient permissions
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate resource
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unexpected failure
503	Service Unavailable	Service down

18.2 API Versioning

```
Strategy 1: URL Path (most common)
  GET /api/v1/orders
  GET /api/v2/orders

Strategy 2: Header
  GET /api/orders
  Header: Accept: application/vnd.myapp.v2+json

Strategy 3: Query Parameter
  GET /api/orders?version=2
```

Recommendation: Use **URL path versioning** for simplicity and cacheability.

18.3 Error Handling Strategy

```java
// Global exception handler
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiResponse<Void>>
handleNotFound(ResourceNotFoundException ex) {
```

```
        return ResponseEntity.status(404).body(
            ApiResponse.error(ex.getMessage(), "RESOURCE_NOT_FOUND"));
    }

    @ExceptionHandler(BusinessException.class)
    public ResponseEntity<ApiResponse<Void>> handleBusiness(BusinessException
ex) {
        return ResponseEntity.status(422).body(
            ApiResponse.error(ex.getMessage(), ex.getErrorCode()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiResponse<Map<String, String>>> handleValidation(
            MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(e ->
            errors.put(e.getField(), e.getDefaultMessage()));
        return ResponseEntity.status(400).body(
            ApiResponse.error("Validation failed", "VALIDATION_ERROR",
errors));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiResponse<Void>> handleGeneral(Exception ex) {
        log.error("Unexpected error", ex);
        return ResponseEntity.status(500).body(
            ApiResponse.error("Internal server error", "INTERNAL_ERROR"));
    }
}
```

18.4 Database Design Rules

Rule	Description
Each service owns its DB	Never access another service's database directly
Use Flyway for migrations	Version control your schema changes
Don't use ddl-auto in prod	Use `validate` or `none`
Index frequently queried columns	Profile and optimize slow queries
Use connection pooling	HikariCP with appropriate pool sizes
Read replicas for read-heavy services	Separate write and read traffic

18.5 General Best Practices

Practice	Why
Design around business capabilities	Services align with domain boundaries
Keep services small and focused	Easier to understand, test, and deploy
Stateless services	Enables horizontal scaling
Config externalization	Change behavior without redeployment
Health checks	Kubernetes knows when to restart/route traffic
Graceful shutdown	Finish in-flight requests before stopping
Idempotent operations	Safe to retry on failure
Contract-first API design	Define API before implementation
Independent deployment	Deploy services without coordinating with other teams
Automated CI/CD	Every push is tested and deployed automatically

---

Part 19: Interview Questions (100+)

Fundamentals (Q1-Q20)

Q1. What are microservices? An architectural style where an application is built as a collection of small, independent, loosely coupled services, each running in its own process and communicating over lightweight protocols like HTTP or messaging.

Q2. Monolith vs Microservices? Monolith: single deployable unit, shared database, one tech stack. Microservices: independently deployed services, database per service, polyglot technology. Monolith is simpler; microservices scale better and enable team autonomy.

Q3. When should you NOT use microservices? Small teams (< 5), simple domains, startups/MVPs, unclear domain boundaries, no DevOps maturity. Microservices add complexity that must be justified by scale/team size.

Q4. What is bounded context? A DDD concept where each service owns a specific business domain with clear boundaries. Example: "Order" in Order Service vs "Order" in Shipping Service may have different attributes.

Q5. How do microservices communicate? Synchronous: REST APIs, gRPC, Feign Client. Asynchronous: Message brokers (Kafka, RabbitMQ). Choose sync when response is needed immediately; async for fire-and-forget.

Q6. What is an API Gateway? Single entry point for all clients. Handles routing, authentication, rate limiting, load balancing, and request aggregation. Examples: Spring Cloud Gateway, Kong, AWS API Gateway.

Q7. What is service discovery? A mechanism for services to register themselves and discover other services dynamically. Eliminates hardcoded URLs. Examples: Eureka, Consul, Kubernetes DNS.

Q8. What is a Config Server? Centralizes configuration for all services. Services fetch config on startup from a Git-backed config server. Enables changing config without redeployment.

Q9. What are the 12-Factor App principles relevant to microservices? Codebase (one per service), dependencies (declared explicitly), config (in environment), backing services (attached resources), build-release-run (strict separation), processes (stateless), port binding (self-contained), concurrency (scale out), disposability (fast startup/shutdown), dev-prod parity, logs (event streams), admin processes.

Q10. What is Domain-Driven Design (DDD)? A design approach where software models are tightly aligned with business domains. Key concepts: bounded contexts, aggregates, entities, value objects, domain events. Essential for defining microservice boundaries.

Q11. How to decompose a monolith into microservices?

1. Identify bounded contexts using DDD. 2. Start with clear domain boundaries. 3. Use Strangler pattern for gradual migration. 4. Extract one service at a time. 5. Ensure independent deployability.

Q12. What is the difference between orchestration and choreography? Orchestration: a central coordinator (orchestrator) directs the workflow. Choreography: each service reacts to events independently without a central controller. Orchestration is simpler to understand; choreography is more decoupled.

Q13. What is eventual consistency? Data across services may be temporarily inconsistent but will converge to a consistent state over time. Opposite of strong consistency (ACID transactions).

Q14. What is a sidecar pattern? Deploying a helper container alongside the main container in the same pod. The sidecar handles cross-cutting concerns like logging, mTLS, traffic management (Envoy proxy).

Q15. What is service mesh? Infrastructure layer that handles service-to-service communication, including load balancing, encryption, observability. Examples: Istio, Linkerd. Uses sidecar proxies.

Q16. What is contract testing? Testing that verifies the API contract between consumer and provider services. Tools: Pact. Ensures changes in one service don't break others.

Q17. What is the strangler fig pattern? Gradually replacing monolith modules with microservices. Route requests to new services via API Gateway while keeping monolith for unchanged functionality.

Q18. What is the backend for frontend (BFF) pattern? A dedicated API Gateway per frontend type (web, mobile, IoT). Each BFF aggregates and transforms data specifically for its frontend.

Q19. How do you handle shared libraries across services? Keep them minimal. Use versioned Maven/Gradle dependencies. Avoid tight coupling through shared domain models. Prefer duplication over coupling.

Q20. What is the difference between microservices and SOA? SOA: enterprise-wide services using ESB (Enterprise Service Bus), SOAP, centralized governance. Microservices: smaller services, REST/messaging, decentralized governance, DevOps-oriented.

Communication & Data (Q21-Q40)

Q21. REST vs gRPC for inter-service communication? REST: text-based (JSON), human-readable, HTTP/1.1, widely supported. gRPC: binary (protobuf), faster, HTTP/2, streaming support. Use gRPC for internal high-performance calls; REST for public APIs.

Q22. What is OpenFeign? Declarative REST client by Spring Cloud. Define an interface with annotations; Spring generates the HTTP client implementation. Supports load balancing and circuit breakers.

Q23. How does Kafka differ from RabbitMQ? Kafka: distributed log, high throughput, ordered within partition, persistent storage, consumer pull. RabbitMQ: traditional message queue, push-based, flexible routing, lower throughput. Use Kafka for event streaming; RabbitMQ for task queues.

Q24. What is the Outbox Pattern? Write business data and event to the database in the same transaction (outbox table). A separate process reads the outbox and publishes to the message broker. Guarantees at-least-once event delivery.

Q25. What is idempotency and why is it important? An operation is idempotent if executing it multiple times has the same result as once. Critical because network failures cause retries, and duplicate messages are unavoidable. Implement using unique idempotency keys.

Q26. How to handle distributed transactions? Use the Saga pattern (choreography or orchestration). Avoid two-phase commit (2PC) in microservices -- it's slow and creates tight coupling.

Q27. What is CQRS? Command Query Responsibility Segregation. Separate write model (normalized, optimized for writes) from read model (denormalized, optimized for queries). Sync via events.

Q28. What is event sourcing? Store all state changes as a sequence of events rather than current state. Rebuild state by replaying events. Provides complete audit trail and temporal queries.

Q29. What is a dead letter queue? A queue where messages that fail processing are sent after exhausting retries. Allows manual inspection and reprocessing of failed messages.

Q30. How to ensure message ordering in Kafka? Messages within a single partition are ordered. Use a consistent partition key (e.g., orderId) to ensure events for the same entity go to the same partition.

Q31. What is the database per service pattern? Each service has its own dedicated database. No direct cross-service DB access. Services communicate via APIs or events. Enables independent scaling and technology choice.

Q32. How to handle JOINs across services? You can't. Options: 1. API composition (call multiple services and join in code). 2. CQRS (maintain denormalized read model). 3. Data duplication (cache subset of needed data).

Q33. What is API composition? A pattern where a composite service calls multiple downstream services and aggregates results. Used to replace cross-service database JOINs.

Q34. How to implement search across multiple services? Use Elasticsearch. Each service publishes events; a search service consumes events and indexes data in Elasticsearch.

Q35. What is the Circuit Breaker pattern? Prevents cascading failures. States: CLOSED (normal), OPEN (fail fast), HALF_OPEN (test recovery). Implemented with Resilience4j or Hystrix.

Q36. What is bulkhead pattern? Isolate resources (thread pools, connections) per dependency. If one dependency fails, only its resources are affected, not the entire service.

Q37. What is a correlation ID? A unique ID generated at the entry point (API Gateway) and passed through all service calls. Enables tracing a single user request across the entire system.

Q38. How to handle data migration in microservices? Use Flyway or Liquibase per service. Version migrations in Git. Run migrations during deployment. Use backward-compatible changes (add columns, never remove).

Q39. What is the publish-subscribe pattern? A messaging pattern where a publisher sends messages to a topic without knowing subscribers. Multiple subscribers can consume the same message. Kafka topics implement this.

Q40. What is back-pressure in reactive systems? A mechanism where consumers tell producers to slow down when overwhelmed. Prevents out-of-memory errors. Implemented in reactive streams (Project Reactor, RxJava).

Architecture & Design (Q41-Q65)

Q41. How to design a fault-tolerant microservices system? Circuit breakers, retries with exponential backoff, bulkheads, timeouts, fallback mechanisms, health checks, graceful degradation, redundant instances, chaos engineering.

Q42. How to implement rate limiting? At API Gateway level using token bucket or sliding window algorithm. Store counters in Redis for distributed rate limiting. Spring Cloud Gateway has built-in support.

Q43. How to handle API versioning? URL path (`/api/v1/...`), custom header (`Accept-Version: v2`), or content negotiation (`Accept: application/vnd.app.v2+json`). URL path is simplest and most common.

Q44. How to implement health checks? Spring Boot Actuator provides `/actuator/health` . Configure liveness probe (is the app alive?) and readiness probe (can it handle traffic?) for Kubernetes.

Q45. What is graceful shutdown? When a service receives SIGTERM, it stops accepting new requests, finishes in-flight requests, and shuts down cleanly. Configure with `server.shutdown=graceful` in Spring Boot.

Q46. How to handle configuration across environments? Spring Cloud Config Server backed by Git. Profile-specific files (`application-dev.yml` , `application-prod.yml`). Secrets via Vault or Kubernetes Secrets.

Q47. How to implement distributed caching? Redis cluster as shared cache. Use `@Cacheable` for read caching, `@CacheEvict` for invalidation. Consider cache-aside pattern for consistency.

Q48. What is a service registry? A database of available service instances and their locations. Services register on startup and deregister on shutdown. Clients query the registry to discover services.

Q49. How to handle cross-cutting concerns? Use API Gateway for external concerns (auth, rate limiting). Use Spring AOP or sidecar proxies for internal concerns (logging, metrics, tracing).

Q50. What is the Ambassador pattern? A proxy that runs alongside a service and handles outgoing connections. Similar to sidecar but specifically for outbound traffic routing, retries, and circuit breaking.

Q51. How to design microservices for high availability? Multiple instances per service, load balancing, health checks, auto-scaling, multi-AZ deployment, circuit breakers, graceful degradation.

Q52. What is a canary deployment? Deploy new version to a small percentage of instances (5%). Monitor for errors. If healthy, gradually increase to 100%. Rollback if issues detected.

Q53. What is blue-green deployment? Maintain two identical environments (blue = current, green = new). Switch traffic from blue to green after testing. Instant rollback by switching back.

Q54. How to handle secrets in microservices? HashiCorp Vault, Kubernetes Secrets, AWS Secrets Manager. Never hardcode secrets. Rotate regularly. Inject via environment variables.

Q55. What is the decompose by business capability pattern? Define services based on business functions (orders, payments, shipping) rather than technical layers (database service, email service).

Q56. What is the decompose by subdomain pattern? Use DDD subdomains: core domain (critical business logic), supporting domain (necessary but not differentiating), generic domain (commodity functions like email, auth).

Q57. How to test microservices? Unit tests (per class), integration tests (per service with real DB), contract tests (API contracts between services), end-to-end tests (full system), chaos tests (failure scenarios).

Q58. What is the testing pyramid for microservices? Many unit tests at the base, fewer integration tests in the middle, minimal end-to-end tests at the top. Emphasize contract testing between services.

Q59. How to implement a distributed lock? Redis `SETNX` with expiry (Redisson). Use when only one service instance should execute a task (e.g., scheduled job in a cluster).

Q60. What is the External Configuration Store pattern? Store configuration in a centralized, external system (Config Server, Consul KV, AWS Parameter Store) rather than in application files.

Q61. How to handle database schema changes without downtime? Use expand-contract pattern: 1. Add new column (backward compatible). 2. Deploy code that writes to both old and new. 3. Migrate data. 4. Remove old column.

Q62. What is the Retry pattern best practices? Retry only transient failures (timeouts, 503). Use exponential backoff (1s, 2s, 4s). Add jitter to prevent thundering herd. Set max retry limit.

Q63. How to implement request aggregation? API Gateway or BFF service calls multiple downstream services, combines results, and returns a single response to the client. Reduces client-side complexity.

Q64. What is the Anti-Corruption Layer (ACL)? A translation layer between a new microservice and a legacy system. Prevents legacy concepts from leaking into the new service's domain model.

Q65. How to handle service versioning in production? Support at least N-1 versions. Deprecate with advance notice. Use API version headers. Monitor which versions clients are using. Remove old versions after migration period.

Production & Operations (Q66-Q85)

Q66. How to monitor microservices in production? Metrics (Prometheus + Grafana), logs (ELK Stack), traces (Zipkin/Jaeger), alerts (PagerDuty/OpsGenie), dashboards per service and business metrics.

Q67. What SLIs, SLOs, and SLAs are? SLI (Service Level Indicator): measurable metric (latency, availability). SLO (Objective): target for SLI (99.9% uptime). SLA (Agreement): contractual commitment based on SLOs.

Q68. How to handle cascading failures? Circuit breaker on every inter-service call. Timeout on all external calls. Bulkhead isolation. Fallback responses. Graceful degradation.

Q69. What is chaos engineering? Deliberately introducing failures to test system resilience. Tools: Netflix Chaos Monkey, Litmus Chaos. Practices: kill random instances, inject network latency, simulate region failure.

Q70. How to scale microservices? Horizontal scaling (more instances). Kubernetes HPA based on CPU/memory/custom metrics. Scale services independently based on load patterns.

Q71. How to handle log aggregation? All services emit structured JSON logs. Collect via Fluentd/Filebeat. Store in Elasticsearch. Query via Kibana. Include correlationId, service name, trace ID in every log.

Q72. What is distributed tracing? Tracking a request as it flows through multiple services. Each service generates a span; all spans in a request share a trace ID. Tools: Zipkin, Jaeger, OpenTelemetry.

Q73. How to implement auto-scaling? Kubernetes HPA watches metrics (CPU, memory, custom). When threshold exceeded, adds pods. When load decreases, removes pods. Configure min/max replicas.

Q74. How to implement zero-downtime deployment? Rolling update (replace instances gradually). Readiness probes (only route traffic to ready instances). Graceful shutdown (finish in-flight requests). Database backward compatibility.

Q75. How to handle network partitions? Accept that partitions happen (CAP theorem). Design for eventual consistency. Use retry with backoff. Implement compensating transactions. Monitor network health.

Q76. What metrics should every microservice expose? Request rate, error rate, latency (p50, p95, p99), JVM metrics (heap, GC), connection pool stats, custom business metrics.

Q77. How to implement feature flags in microservices? Use tools like LaunchDarkly, Unleash, or Spring Cloud Config. Store flags externally. Evaluate at runtime. Enable gradual rollouts without deployment.

Q78. How to handle data backup in microservices? Each service responsible for its own database backup. Automated backups with point-in-time recovery. Cross-region replication for disaster recovery.

Q79. How to implement a kill switch? Feature flag that disables a feature instantly without deployment. Useful for disabling a misbehaving feature in production.

Q80. What is the importance of health check endpoints? Kubernetes uses liveness probes to restart unhealthy pods and readiness probes to stop routing traffic to unready pods. Without them, dead pods receive traffic.

Q81. How to handle service discovery failure? Cache last known service locations. Use DNS-based discovery as fallback. Multiple Eureka servers for HA. Kubernetes DNS is highly available by default.

Q82. How to implement log-based alerting? ELK + ElastAlert: define rules like "alert if ERROR count > 10 in 5 minutes for service X." Send alerts to Slack, PagerDuty, email.

Q83. How to handle memory leaks in containerized microservices? Set JVM heap limits matching container limits. Monitor with Prometheus JVM exporter. Use `-XX:+ExitOnOutOfMemoryError` flag. Kubernetes restarts OOM-killed pods.

Q84. How to implement request tracing across async (Kafka) boundaries? Pass trace ID in Kafka message headers. Consumer extracts trace ID and sets it as parent span. This links producer and consumer traces in Zipkin.

Q85. What is the difference between liveness and readiness probes? Liveness: "Is the process alive?" Restart if not. Readiness: "Can it handle traffic?" Stop routing traffic if not. A service can be alive but not ready (still starting up).

Architect-Level Questions (Q86-Q105)

Q86. Design a microservices architecture for an e-commerce platform. Services: User, Product, Order, Payment, Inventory, Notification, Search, Review. API Gateway for routing and auth. Kafka for events. Redis for caching. Saga for order processing.

Q87. How would you migrate a monolith to microservices?

1. Identify bounded contexts (DDD). 2. Extract one service at a time (Strangler pattern). 3. Use API Gateway to route between monolith and services. 4. Share data via events, not DB. 5. CI/CD per service.

Q88. Design an event-driven order processing system. Order Service creates order, publishes OrderCreated. Inventory Service reserves stock, publishes StockReserved. Payment Service charges, publishes PaymentProcessed. On failure, compensating events undo operations (Saga).

Q89. How to handle data consistency across microservices? Accept eventual consistency. Use Saga pattern for distributed transactions. Outbox pattern for reliable event publishing. Idempotent consumers for safe retries.

Q90. Design a caching strategy for a product catalog service. Cache products in Redis (cache-aside). TTL of 1 hour. Invalidate on product update events. Use read-through for popular items. Multi-level cache (local + distributed).

Q91. How to implement multi-tenant microservices? Tenant ID in JWT claims. Filter data in every query by tenant ID. Separate databases per tenant (highest isolation) or shared database with tenant column.

Q92. Design a notification system as a microservice. Kafka consumer for events. Template engine for messages. Provider abstraction (email, SMS, push). Retry queue for failed deliveries. User preferences for notification channels.

Q93. How to handle service discovery in Kubernetes vs Eureka? Kubernetes: built-in DNS-based discovery, no extra infrastructure. Eureka: works outside Kubernetes, more features (metadata, health checks). In K8s, use native discovery.

Q94. Design a CI/CD pipeline for microservices. Each service has its own pipeline. Stages: build, unit test, integration test, Docker build, push to registry, deploy to staging, run smoke tests, deploy to production (canary).

Q95. How to implement a distributed scheduler in microservices? Use ShedLock with Redis/DB to ensure only one instance runs a scheduled task. Or use Kubernetes CronJob for stateless batch tasks.

Q96. Design security for a microservices system. API Gateway validates JWT. Auth Service issues tokens. Services extract user info from headers (set by Gateway). RBAC per endpoint. mTLS for service-to-service. Secrets in Vault.

Q97. How to handle API rate limiting across instances? Use distributed counter in Redis. Token bucket or sliding window algorithm. Rate limit key: client IP, API key, or user ID. Return 429 with Retry-After header.

Q98. Design a monitoring and alerting architecture. Metrics: Micrometer -> Prometheus -> Grafana. Logs: Logback -> Fluentd -> Elasticsearch -> Kibana. Traces: Micrometer Tracing -> Zipkin. Alerts: Grafana/Prometheus Alertmanager -> PagerDuty.

Q99. How to implement a saga with compensating transactions? Orchestrator calls services sequentially. On failure at any step, call compensating actions for all completed steps in reverse order. Log all saga steps for debugging.

Q100. Design a real-time analytics pipeline with microservices. Business services publish events to Kafka. Stream processing (Kafka Streams/Flink) aggregates in real-time. Store in time-series DB (InfluxDB) or Elasticsearch. Visualize in Grafana.

Q101. How to handle database migration in a blue-green deployment? Use expand-contract: add new columns before deploying new code. Both old and new code work with the schema. After full migration, remove old columns.

Q102. Design a content delivery system using microservices. Upload Service -> Object Storage (S3). CDN (CloudFront) for delivery. Metadata Service for file metadata. Thumbnail Service for image processing. Event-driven processing pipeline.

Q103. How to implement request deduplication? Client sends idempotency key in header. Service stores key -> response mapping in Redis. If duplicate key, return cached response. TTL on keys prevents indefinite storage.

Q104. Design a real-time chat system using microservices. Connection Service (WebSocket management). Message Service (store and deliver). Presence Service (online status). Notification Service (push for offline users). Kafka for message routing.

Q105. How would you evaluate whether to break a monolith into microservices? Evaluate: team size (> 8 people?), deployment frequency (weekly?), scaling needs (different components?), technology constraints, domain complexity. Calculate cost of distributed system complexity vs benefits of independence.

Part 20: Hands-on Exercises

Exercise 1: Build an API Gateway

Build a Spring Cloud Gateway that:

- Routes `/api/users/**` to user-service
- Routes `/api/orders/**` to order-service
- Validates JWT on every request
- Returns 401 for invalid/missing tokens

```java
@SpringBootApplication
public class GatewayApp {
    public static void main(String[] args) {
        SpringApplication.run(GatewayApp.class, args);
    }
}
```

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: user-route
          uri: http://localhost:8081
          predicates:
            - Path=/api/users/**
        - id: order-route
          uri: http://localhost:8082
          predicates:
            - Path=/api/orders/**
```

Exercise 2: Implement Service Discovery

1. Create a Eureka Server on port 8761
2. Register two services (user-service, order-service)
3. Use service names for inter-service calls

```yaml
# user-service/application.yml
spring:
  application:
    name: user-service
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

Exercise 3: Implement Kafka Messaging

Producer (Order Service):

```java
@Service
public class OrderEventProducer {
    @Autowired private KafkaTemplate<String, OrderEvent> kafka;

    public void publishOrderCreated(Order order) {
        kafka.send("order-events", order.getId().toString(),
            new OrderEvent("ORDER_CREATED", order.getId(),
                order.getCustomerId(), order.getTotalAmount()));
    }
}
```

Consumer (Notification Service):

```java
@KafkaListener(topics = "order-events", groupId = "notification-group")
public void handleOrderEvent(OrderEvent event) {
    if ("ORDER_CREATED".equals(event.getType())) {
        emailService.sendOrderConfirmation(event.getCustomerId());
    }
}
```

Exercise 4: Implement Redis Caching

```java
@Configuration
@EnableCaching
```

```java
public class RedisConfig {
    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory factory) {
        return RedisCacheManager.builder(factory)
            .cacheDefaults(RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(30)))
            .build();
    }
}

@Service
public class ProductService {
    @Cacheable(value = "products", key = "#id")
    public Product getProduct(Long id) {
        return productRepository.findById(id).orElseThrow();
    }

    @CacheEvict(value = "products", key = "#id")
    public Product updateProduct(Long id, ProductRequest req) {
        Product p = productRepository.findById(id).orElseThrow();
        p.setName(req.getName());
        return productRepository.save(p);
    }
}
```

Exercise 5: Implement Circuit Breaker

```java
@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
@Retry(name = "paymentService")
public PaymentResponse processPayment(PaymentRequest request) {
    return paymentClient.process(request);
}

private PaymentResponse paymentFallback(PaymentRequest req, Throwable t) {
    log.warn("Payment service down: {}", t.getMessage());
    return new PaymentResponse("QUEUED", "Will process later");
}
```

Exercise 6: Implement Feign Client with Fallback

```java
@FeignClient(name = "inventory-service",
             fallbackFactory = InventoryFallbackFactory.class)
public interface InventoryClient {
    @GetMapping("/api/inventory/{productId}")
    InventoryResponse getStock(@PathVariable Long productId);

    @PostMapping("/api/inventory/reserve")
    InventoryResponse reserveStock(@RequestBody ReserveRequest request);
}

@Component
public class InventoryFallbackFactory
        implements FallbackFactory<InventoryClient> {
    @Override
    public InventoryClient create(Throwable cause) {
        return new InventoryClient() {
            @Override
            public InventoryResponse getStock(Long productId) {
                return new InventoryResponse(productId, -1, "UNKNOWN");
            }
            @Override
            public InventoryResponse reserveStock(ReserveRequest req) {
                throw new ServiceUnavailableException("Inventory service
down");
            }
        };
    }
}
```

Exercise 7: Implement Saga Orchestrator

```java
@Service
public class OrderSaga {
    public OrderResult execute(OrderRequest request) {
        Order order = createOrder(request);              // Step 1
        try {
            reserveInventory(order);                     // Step 2
            processPayment(order);                       // Step 3
            confirmOrder(order);                         // Step 4
            return OrderResult.success(order);
        } catch (InventoryException e) {
            cancelOrder(order, "Insufficient stock");    // Compensate 1
```

```
            return OrderResult.failed(order);
        } catch (PaymentException e) {
            releaseInventory(order);                    // Compensate 2
            cancelOrder(order, "Payment failed");       // Compensate 1
            return OrderResult.failed(order);
        }
    }
}
```

Exercise 8: Implement Health Checks

```
@Component
public class DatabaseHealthIndicator implements HealthIndicator {
    @Autowired private DataSource dataSource;

    @Override
    public Health health() {
        try (Connection conn = dataSource.getConnection()) {
            return Health.up()
                .withDetail("database", "Reachable")
                .build();
        } catch (Exception e) {
            return Health.down()
                .withException(e)
                .build();
        }
    }
}
```

Exercise 9: Implement Distributed Tracing

```
# application.yml
management:
  tracing:
    sampling:
      probability: 1.0
  zipkin:
```

```
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
```

Exercise 10: Docker Compose for Full System

```yaml
version: '3.8'
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    ports: ["5432:5432"]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    ports: ["9092:9092"]

  zipkin:
    image: openzipkin/zipkin
    ports: ["9411:9411"]

  api-gateway:
    build: ./api-gateway
    ports: ["8080:8080"]

  user-service:
    build: ./user-service
    depends_on: [postgres, redis]

  order-service:
    build: ./order-service
    depends_on: [postgres, kafka, redis]
```

End of Guide

Document Version: 1.0 **Last Updated:** March 2026 **Topics Covered:** 20 Parts, 105 Interview Questions, 10 Hands-on Exercises